explain me python to create json files. need to create python scripts needed for spark streaming.
then create producers and consumers in apache kafka

Yes. Since your goal is **Spark Structured Streaming + Kafka**, you don't need to learn all of Python first. You mainly need the Python skills for:

1. Creating JSON data
2. Writing JSON files
3. Generating new JSON events continuously
4. Reading JSON events
5. Using Python with Kafka producers
6. Using Python with Kafka consumers

We'll learn it **step by step**, from very basic Python to a small Kafka → Spark project.

Roadmap

```
Python basics
    ↓
Dictionaries
    ↓
Lists
    ↓
JSON with Python
    ↓
Generate JSON files
    ↓
Generate JSON events continuously
    ↓
Kafka Python Producer
    ↓
Kafka Python Consumer
    ↓
Kafka → Spark Structured Streaming
    ↓
Transform → Output
```

Part 1 — Python data structures you need

1. Variables

Python Run

```python
name = "Youssef"
price = 100
quantity = 3

total = price * quantity

print(name)
print(total)
```

Output:

```
Youssef
300
```

For streaming data, you'll frequently have values like:

Python Run

```python
order_id = 101
product = "Laptop"
price = 1200
quantity = 2
```

---

2. Python dictionaries

This is **very important** because a JSON object looks almost exactly like a Python dictionary.

Python Run

```python
order = {
    "order_id": 101,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}

print(order)
```

You can access individual fields:

Python

Run

```python
print(order["order_id"])
print(order["product"])
print(order["price"])
```

Output:

```
101
Laptop
1200
```

Think:

```
Python dictionary
        ↓
{
    key: value
}
```

---

3. Lists

A list contains multiple values.

Python

Run

```python
products = [
    "Laptop",
    "Phone",
    "Keyboard"
]

print(products)
```

You can also have multiple dictionaries:

Python

Run

```
orders = [
    {
        "order_id": 101,
        "product": "Laptop",
        "price": 1200
    },
    {
        "order_id": 102,
        "product": "Phone",
        "price": 800
    },
    {
        "order_id": 103,
        "product": "Keyboard",
        "price": 100
    }
]
```

This structure is extremely useful when creating test data.

---

4. What is JSON?

JSON is simply a text format for representing data.

Example:

JSON

```
{
    "order_id": 101,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}
```

Python has a built-in module called `json` .

Python Run

```
import json
```

---

5. Python dictionary → JSON

Python Run

```python
import json

order = {
    "order_id": 101,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}

json_data = json.dumps(order)

print(json_data)
```

Output:

```
{"order_id": 101, "product": "Laptop", "price": 1200, "quantity": 2}
```

The important function is:

Python Run

```python
json.dumps()
```

It means:

```
Python object
      ↓
JSON string
```

---

6. Write JSON to a file

Now let's create an actual JSON file.

Create:

```
create_json.py
```

Code:

Python Run

```python
import json

order = {
    "order_id": 101,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}

with open("order.json", "w") as file:
    json.dump(order, file, indent=4)

print("JSON file created.")
```

Run:

Bash

```bash
python3 create_json.py
```

You'll get:

```
order.json
```

Containing:

JSON

```json
{
    "order_id": 101,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}
```

---

7. Creating multiple JSON records

For Spark Streaming practice, you'll usually want multiple events.

Python Run

```python
import json

orders = [
    {
        "order_id": 101,
        "product": "Laptop",
        "price": 1200,
        "quantity": 2
    },
    {
        "order_id": 102,
        "product": "Phone",
        "price": 800,
        "quantity": 1
    },
    {
        "order_id": 103,
        "product": "Keyboard",
        "price": 100,
        "quantity": 3
    }
]

with open("orders.json", "w") as file:
    json.dump(orders, file, indent=4)

print("Orders created.")
```

The file becomes:

JSON

```
[
    {
        "order_id": 101,
        "product": "Laptop",
        "price": 1200,
        "quantity": 2
    },
    {
        "order_id": 102,
        "product": "Phone",
        "price": 800,
        "quantity": 1
    },
    {
        "order_id": 103,
        "product": "Keyboard",
        "price": 100,
        "quantity": 3
    }
]
```

---

8. But Spark Streaming has an important pattern

For your Spark Structured Streaming practice, we want to simulate events arriving over time.

Instead of:

```
orders.json
    ↓
Spark
```

we'll eventually do:

```
Python script
     ↓
new JSON event
     ↓
HDFS directory
     ↓
Spark Structured Streaming
```

For example:

```
/data/json/
    event_001.json
    event_002.json
    event_003.json
    event_004.json
```

Spark watches the directory.

When Python creates:

```
event_005.json
```

Spark detects it.

---

9. Generate JSON events continuously

Now we need a little more Python.

We'll use:

Python Run

```python
import json
import time
import random
```

Example:

Python Run

```python
import json
import time
import random

products = ["Laptop", "Phone", "Keyboard", "Mouse"]

order_id = 1

while True:

    order = {
        "order_id": order_id,
        "product": random.choice(products),
        "price": random.randint(50, 1500),
        "quantity": random.randint(1, 5)
    }

    filename = f"event_{order_id}.json"

    with open(filename, "w") as file:
        json.dump(order, file)

    print(order)

    order_id += 1

    time.sleep(5)
```

Now Python generates:

```
event_1.json
event_2.json
event_3.json
event_4.json
...
```

every 5 seconds.

This is already a very useful **Spark Structured Streaming producer simulation**.

---

10. Add event time

For streaming, timestamps are very important.

Python:

```python
from datetime import datetime
```

Then:

```python
"event_time": datetime.now().isoformat()
```

Example:

```python
{
    "order_id": 1,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2,
    "event_time": "2026-08-29T20:20:15.123456"
}
```

This becomes extremely useful later when you learn:

- event time
- windows
- watermarks
- late data

---

11. Reading JSON with Python

You'll also need to read JSON.

```python
import json

with open("order.json", "r") as file:
    order = json.load(file)

print(order)
```

Now `order` is a Python dictionary again.

```python
```

```
print(order["product"])
```

So remember:

```
json.dump()
    Python → JSON file

json.dumps()
    Python → JSON string

json.load()
    JSON file → Python

json.loads()
    JSON string → Python
```

This distinction is important for Kafka.

---

12. Why Kafka is slightly different

With files:

```
Python
   ↓
JSON file
   ↓
Spark
```

With Kafka:

```
Python Producer
       ↓
      Kafka
       ↓
Python Consumer
```

And eventually:

```
Python Producer
       ↓
      Kafka
       ↓
Spark Structured Streaming
       ↓
Transformations
       ↓
HDFS / Parquet / Delta
```

The Kafka producer sends messages rather than creating files.

---

13. Kafka Producer

For Python Kafka work, you'll install a Kafka client library.

One common choice is:

Bash

```
pip install kafka-python
```

Then:

Python Run

```python
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda value: json.dumps(value).encode("utf-8")
)
```

Now create an event:

Python Run

```python
order = {
    "order_id": 1,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}
```

Send it:

Python Run

```python
producer.send("orders", value=order)
```

Then:

Python Run

```python
producer.flush()
```

Complete basic producer:

Python Run

```python
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda value: json.dumps(value).encode("utf-8")
)

order = {
    "order_id": 1,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}

producer.send("orders", value=order)

producer.flush()

print("Message sent.")
```

Conceptually:

```
Python dictionary
      ↓
json.dumps()
      ↓
JSON string
      ↓
UTF-8 bytes
      ↓
Kafka
```

---

14. Kafka Consumer

The consumer receives the message.

Python

```python
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "orders",
    bootstrap_servers="localhost:9092",
    auto_offset_reset="earliest",
    value_deserializer=lambda value: json.loads(value.decode("utf-8"))
)

for message in consumer:
    print(message.value)
```

The result is again a Python dictionary:

Python

```python
{
    "order_id": 1,
    "product": "Laptop",
    "price": 1200,
    "quantity": 2
}
```

So you have:

```
Producer
   ↓
Python dictionary
   ↓
JSON
   ↓
Kafka
   ↓
JSON
   ↓
Python dictionary
   ↓
Consumer
```

---

15. Your eventual project

I recommend we build this progressively:

Stage 1 — Python JSON

```
create_json.py
        ↓
orders.json
```

Stage 2 — JSON event generator

```
generate_events.py
        ↓
event_1.json
event_2.json
event_3.json
...
        ↓
Spark Structured Streaming
```

Stage 3 — Kafka producer

```
producer.py
     ↓
Kafka topic: orders
```

Stage 4 — Kafka consumer

```
Kafka topic
     ↓
consumer.py
```

Stage 5 — Kafka + Spark

```
Python Producer
      ↓
Kafka
      ↓
Spark Structured Streaming
      ↓
JSON parsing
      ↓
Transformations
      ↓
Aggregations
      ↓
Watermark
      ↓
Window
      ↓
HDFS / Parquet
```

Stage 6 — Full data-engineering project

```
Python Event Generator
          ↓
      Kafka
          ↓
Spark Structured Streaming
          ↓
       Bronze
          ↓
       Silver
          ↓
        Gold
          ↓
     Parquet/Delta
```

For you, I would start with Stage 1 and not jump into Kafka yet. Master about 10–15 small Python exercises involving dictionaries, lists, `json`, files, loops, `random`,

`datetime` , and functions. Then the Kafka producer/consumer code will be much easier to understand.